Part I

Foundations of data management

Plan

2. First Order Logic (FOL)
   - Basic notions
3. What is a data model?
   - General FOL-based data model
   - Example: the relational data model
4. What is a query language?
   - FOL queries
   - Example: the relational conjunctive queries
5. What are the data management problems?
   - Consistency checking & query answering

FOL formula

Given a set $\mathcal{P}$ of predicates and a set $\mathcal{C}$ of constants, a (function-free) *FOL formula* ϕ is inductively defined as:

- the constants `false` (a.k.a. $\perp$) and `true` (a.k.a. $\top$)
- an atom $R(t_1, \ldots, t_n)$ whose terms $t_{1 \leq i \leq n}$ are constants or variables
- a negation of a formula $(\neg \phi_1)$
- a conjunction of formulae $(\phi_1 \wedge \phi_2)$
- a disjunction of formulae $(\phi_1 \vee \phi_2)$
- an implication between formulae $(\phi_1 \Rightarrow \phi_2)$
- an equivalence of formulae $(\phi_1 \Leftrightarrow \phi_2)$
- an existential quantification of a formula $\exists \bar{x}\ \phi_1(\bar{x})$
- a universal quantification of a formula $\forall \bar{x}\ \phi_1(\bar{x})$

where $\phi(\bar{x})$ means that every variable in the set $\bar{x}$ has some *free* occurrence in ϕ.

Closed versus open FOL formulae

A closed FOL formula has all its variables bound; an open FOL formula has free variables.

Interpretation/semantics (i.e., meaning) of a closed FOL formula

The semantics of a closed FOL formula ϕ is given in terms of a *logical interpretation* $I = \langle \Delta^I, .^I \rangle$, with Δ^I the *interpretation domain* and $.^I$ the *interpretation function* that assigns (i) elements of Δ^I to constants in $\mathcal{C}$, i.e., $a^I \in \Delta^I$ for a constant $a \in \mathcal{C}$, and (ii) tuples from $(\Delta^I)^n$ to predicates in $\mathcal{P}$, i.e., $P^I \subseteq (\Delta^I)^n$ for a predicate $P/n \in \mathcal{P}$. The interpretation of a FOL formula ϕ is inductively defined as:

- $[\texttt{false}]^I = \texttt{false}$ and $[\texttt{true}]^I = \texttt{true}$
- $[R(t_1, \ldots, t_n)]^I = \texttt{true}$ iff $(t_1^I, \ldots, t_n^I) \in R^I$
- $[(\neg\phi_1)]^I = \neg[\phi_1]^I$
- $[(\phi_1 \wedge \phi_2)]^I = [\phi_1]^I \wedge [\phi_2]^I$
- $[(\phi_1 \vee \phi_2)]^I = [\phi_1]^I \vee [\phi_2]^I$
- $[(\phi_1 \Rightarrow \phi_2)]^I = [\phi_1]^I \Rightarrow [\phi_2]^I$
- $[(\phi_1 \Leftrightarrow \phi_2)]^I = [\phi_1]^I \Leftrightarrow [\phi_2]^I$
- $[\exists \bar{x}\ \phi_1(\bar{x})]^I = \texttt{true}$ iff $\exists \bar{t} \in (\Delta^I)^{|\bar{x}|}$, $[\phi_1(\bar{t})]^I = \texttt{true}$
- $[\forall \bar{x}\ \phi_1(\bar{x})]^I = \texttt{true}$ iff $\forall \bar{t} \in (\Delta^I)^{|\bar{x}|}$, $[\phi_1(\bar{t})]^I = \texttt{true}$

Model of a FOL formula

An interpretation that makes a FOL formula `true` is a *model* of this formula.

FOL formulae can be compared wrt a generalization/specialization relation:

Logical consequence relation ($\models$)

A FOL formula ϕ_1 is a *logical consequence of* (a.k.a. *entailed by*) a FOL formula ϕ_2, denoted $\phi_2 \models \phi_1$, iff every model of ϕ_2 is also a model of ϕ_1.

Example (Interpretation, model, and consequence)

Let I be an interpretation such that: $\Delta^I = \{e_1, e_2\}$, $edge^I = \{(e_1, e_2)\}$, $source^I = \{e_1\}$ and $target^I = \{e_2\}$.

- $[\forall x \forall y (edge(x, y) \Rightarrow source(x))]^I = \texttt{true}$
- $[\forall x \forall y (edge(x, y) \wedge source(x))]^I = \texttt{false}$
- $[\exists x \forall y (edge(x, y) \wedge source(x))]^I = \texttt{false}$
- $[\forall x \exists y (edge(x, y) \wedge source(x))]^I = \texttt{false}$
- $[\exists x \exists y (edge(x, y) \wedge source(x))]^I = \texttt{true}$
- $\forall x \forall y (edge(x, y) \wedge source(x)) \models \exists x \exists y (edge(x, y) \wedge source(x))$

Foundations of FOL data models

Many data models are defined in or can be (almost entirely) translated in FOL.

Data model

A data model is a triple $\langle \mathcal{L}_{\mathcal{F}}, \mathcal{L}_{\mathcal{C}}, \mathcal{A} \rangle$ where:

- $\mathcal{L}_{\mathcal{F}}$ is a FOL language for expressing facts
- $\mathcal{L}_{\mathcal{C}}$ is a FOL language for expressing constraints on facts
- $\mathcal{A}$ is an assumption on the completeness of the stored facts: either the *Closed World Assumption (CWA)* or the *Open World Assumption (OWA)*.

CWA versus OWA

CWA: the stored facts are complete, any other fact is false.
OWA: the stored facts are incomplete, some other facts may be true.

Database

A *database* for a data model $\langle \mathcal{L}_{\mathcal{F}}, \mathcal{L}_{\mathcal{C}}, \mathcal{A} \rangle$ is a *closed logical formula* $\mathcal{E}_{\mathcal{L}_{\mathcal{F}}} \wedge \mathcal{E}_{\mathcal{L}_{\mathcal{C}}}$ interpreted under $\mathcal{A}$, with $\mathcal{E}_{\mathcal{L}_{\mathcal{F}}}$ an expression of $\mathcal{L}_{\mathcal{F}}$ and $\mathcal{E}_{\mathcal{L}_{\mathcal{C}}}$ an expression of $\mathcal{L}_{\mathcal{C}}$.

Importantly, under CWA, if a fact f is **not** stored in a database $\mathcal{D}$, then it does not hold in the database: $\mathcal{D} \models \neg f$.

Foundations of FOL data models

Consistent database

A database is *consistent* iff it has a model, i.e., the data together with constraints are *satisfiable*.

Remark: an inconsistent database is equivalent to `false`.

FOL characterization of the relational data model

The relational model

- $\mathcal{L}_{\mathcal{F}}$ is the language of *conjunctions of logical ground atoms*: $\bigwedge_{i=1}^{n} a_i(cst_i^1, \ldots, cst_i^{l_i})$.
- $\mathcal{L}_{\mathcal{C}}$ is the language of *conjunctions of functional dependencies*

$$\forall \bar{x} \forall \bar{y} \forall \bar{y}' \forall \bar{z} \forall \bar{z}' (R(\bar{x}, \bar{y}, \bar{z}) \wedge R(\bar{x}, \bar{y}', \bar{z}') \Rightarrow \bar{y} = \bar{y}')$$

  and of inclusion dependencies

$$\forall \bar{x} \forall \bar{y} \ (R(\bar{x}, \bar{y}) \Rightarrow \exists \bar{z} \ S(\bar{x}, \bar{z}))$$

- $\mathcal{A}$ is *CWA*

FOL characterization of the relational data model (by example)

Example: from SQL to the relational model

SQL notation	Relational notation

`create table Employee (`
`name varchar(20),`
`team varchar(20),`
`dept varchar(20),`
`primary key (name,team));`
	$Employee(name, team, dept)$
	$Employee : name, team \rightarrow dept$

`create table Salary (`
`name varchar(20),`
`team varchar(20),`
`salary int,`
`primary key (name,team),`
`foreign key (name,team)`
`references Employee(name,team));`
	$Salary(name, team, salary)$
	$Salary : name, team \rightarrow salary$
	$Salary[name, team] \subseteq Employee(name, team)$

`insert into table Employee` `values ('FG','SHAMAN','INFO');`	$Employee(FG, SHAMAN, INFO)$ $Salary(FG, SHAMAN, 1000000)$
`insert into table Salary` `values ('FG','SHAMAN',1000000);`	

FOL characterization of the relational data model (by example)

Example cont.: from the relational model to FOL

Relational notation	FOL notation
$Employee(name, team, dept)$ $Employee : name, team \rightarrow dept$	$\forall name \forall team \forall dept \forall dept'$ $(Employee(name, team, dept)$ $\wedge$ $Employee(name, team, dept')$ $\Rightarrow dept = dept')$
$Salary(name, team, salary)$ $Salary : name, team \rightarrow salary$ $Salary[name, team] \subseteq Employee(name, team)$	$\wedge$ $\forall name \forall team \forall salary \forall salary'$ $(Salary(name, team, salary)$ $\wedge$ $Salary(name, team, salary')$ $\Rightarrow salary = salary')$
$Employee(FG, SHAMAN, INFO)$ $Salary(FG, SHAMAN, 1000000)$	$\wedge$ $\forall name \forall team \forall salary$ $(Salary(name, team, salary)$ $\Rightarrow \exists dept\ Employee(name, team, dept))$ $\wedge$ $Employee(FG, SHAMAN, INFO)$ $\wedge$ $Salary(FG, SHAMAN, 1000000)$

FOL characterization of the relational data model (by example)

Example cont.

This consistent database corresponds to:

Employee	*name*	*team*	*dept*
	FG	*SHAMAN*	*INFO*

Salary	*name*	*team*	*salary*
	FG	*SHAMAN*	1000000

What happens if we insert $Salary(LO, SHAMAN, 1)$ in the database $\mathcal{D}$?
$\mathcal{D}$ becomes inconsistent!
Why? Because of CWA!

Employee	*name*	*team*	*dept*
	FG	*SHAMAN*	*INFO*
	LO	*SHAMAN*	?

Salary	*name*	*team*	*salary*
	FG	*SHAMAN*	1000000
	LO	*SHAMAN*	1

In this case, $\mathcal{D} \models \exists d\ Employee(LO, SHAMAN, d)$ holds.
Also, since there is no stored fact of the form $Employee(LO, SHAMAN, ?)$, then $\mathcal{D} \models \neg(\exists d\ Employee(LO, SHAMAN, d))$ holds because of CWA.
Therefore, $\mathcal{D} \models$ `false` holds too.

Foundations of FOL queries

FOL query

A query is a FOL formula $\phi(\bar{x})$, with $\bar{x}$ the (possibly empty) set of free variables.
$|\bar{x}|$ is the arity of the query.

FOL query interpretation/semantics

The semantics of a FOL query is given in terms of a logical interpretation $I = \langle \Delta^I, .^I \rangle$:

- The interpretation of a Boolean FOL formula $\phi()$ is: `true` iff $[\phi()]^I = \texttt{true}$.
- The interpretation of a FOL formula $\phi(\bar{x})$ is: $\{\bar{t} \in (\Delta^I)^{|\bar{x}|} \mid [\phi(\bar{t})]^I = \texttt{true}\}$.

Example of query semantics

Consider the interpretation $I = \langle \Delta^I, .^I \rangle$ such that: $\Delta^I = \{e_1, e_2\}$, $edge^I = \{(e_1, e_2)\}$, $source^I = \{e_1\}$ and $target^I = \{e_2\}$.

- $[\forall x \forall y (edge(x, y) \Rightarrow source(x))]^I$ is: `true`
- $[\forall y (edge(x, y) \Rightarrow source(x))]^I$ is: $\{e_1, e_2\}$
- $[\forall y (edge(x, y) \wedge source(x))]^I$ is: $\emptyset$
- $[\exists y (edge(x, y) \wedge source(x))]^I$ is: $\{e_1\}$

Foundations of FOL query answers against a database

Answer set of a query against a database

Given a FOL database $\mathcal{D}$ and a FOL query $\phi(\bar{x})$, the answer set of q against $\mathcal{D}$ is:

$$q(\mathcal{D}) = \{\bar{t} \in \mathcal{C}^{|\bar{x}|} \mid \mathcal{D} \models \phi(\bar{t})\}$$

where $\mathcal{C}$ is the set of constants in $\mathcal{D}$.

Remark: if the database is inconsistent, then every tuple is an answer!

(Relational) Conjunctive queries

The central FOL query language is that of *Conjunctive Queries (CQs)*.
A CQ $q(\bar{x})$ is a conjunction of atoms whose terms are either constants, free variables from $\bar{x}$, or existentially quantified variables.

Notations

A FOL query is often noted $q(\bar{x}) \leftarrow \phi(\bar{x})$, where q is the query *name*, $q(\bar{x})$ is the query *head*, and $\phi(\bar{x})$ is the query *body*.

Foundations of FOL query answers against a database

Example: query answers against a database

Consider the consistent database $\mathcal{D}$ **assuming OWA**:

$\forall name \forall team \forall dept \forall dept'$
$(Employee(name, team, dept) \wedge Employee(name, team, dept') \Rightarrow dept = dept')$
$\wedge$
$\forall name \forall team \forall salary \forall salary'$
$(Salary(name, team, salary) \wedge Salary(name, team, salary') \Rightarrow salary = salary')$
$\wedge$
$\forall name \forall team \forall salary$
$(Salary(name, team, salary) \Rightarrow \exists dept\ Employee(name, team, dept))$
$\wedge$
$Employee(FG, SHAMAN, INFO)$
$\wedge$
$Salary(FG, SHAMAN, 1000000)$
$\wedge$
$Salary(LO, SHAMAN, 1)$

What is the answer set of:

- $q_1(x) \leftarrow \exists y\ Employee(x, SHAMAN, y)$ w.r.t. $\mathcal{D}$? $q_1(\mathcal{D}) = \{FG, LO\}$
- $q_2(x_1, x_2) \leftarrow \exists y\ Employee(x_1, y, INFO) \wedge Salary(x_1, y, x_2)$ w.r.t. $\mathcal{D}$?
  $q_2(\mathcal{D}) = \{(FG, 1000000)\}$

Foundations of FOL query answers against a database

Example (cont.): query answers against a database

The following interpretation I is the minimal model of $\mathcal{D}$:

$$\Delta^I = \{FG, SHAMAN, INFO, 1000000, LO, 1, e\}$$

- $FG^I = FG$, $SHAMAN^I = SHAMAN$, $INFO^I = INFO$, $1000000^I = 1000000$, $LO^I = LO$, $1^I = 1$
- $Employee^I = \{(FG, SHAMAN, INFO), (LO, SHAMAN, e)\}$, $Salary^I = \{(FG, SHAMAN, 1000000), (LO, SHAMAN, 1)\}$

The answer set of $q_1(x) \leftarrow \exists y\ Employee(x, SHAMAN, y)$ w.r.t. $\mathcal{D}$ is $q_1(\mathcal{D}) = \{FG, LO\}$ because

- $\mathcal{D} \models q_1(FG)$ and $\mathcal{D} \models q_1(LO)$

The answer set of $q_2(x_1, x_2) \leftarrow \exists y\ Employee(x_1, y, INFO) \wedge Salary(x_1, y, x_2)$ w.r.t. $\mathcal{D}$ is $q_2(\mathcal{D}) = \{FG\}$ because

- $\mathcal{D} \models q_2(FG, 1000000)$

FOL characterization of the relational conjunctive queries (by example)

Example: from SQL to relational algebra, to CQ

Consider the following SQL query:

```
SELECT name, salary
FROM Employee as E, Salary as S
WHERE E.name=S.name AND E.team=S.team and E.dept='INFO';
```

It translates into the relational algebra query:

$$\Pi_{name,salary}(\sigma_{dept=INFO}(Employee \bowtie Salary))$$

and to the CQ:

$$q(x_1, x_2) \leftarrow \exists y\ Employee(x_1, y, INFO) \wedge Salary(x_1, y, x_2)$$

The consistency checking and query answering problems

Consistency checking

Given a database $\mathcal{D}$ of a given data model, the consistency checking problem amounts to deciding whether $\mathcal{D} \not\models$ `false`.

The consistency problem is important in Data Management, as querying an inconsistent database does not make much sense (any tuple is an answer to any query).

In FOL databases, consistency can be checked using refutation

Given a database $\mathcal{D}$, checking the consistency of $\mathcal{D}$ (i.e., $\mathcal{D} \not\models$ `false`) can be done by refuting $\mathcal{D}$: deriving `false` from $\mathcal{D}$ using resolution.
For instance, check that the above database is inconsistent under CWA.

Query answering

Given a consistent database $\mathcal{D}$ of a given data model and a query q, the query answering problem amounts to computing the answer set of q w.r.t. $\mathcal{D}$.

In FOL databases, query answering can be done using refutation

Given a database $\mathcal{D}$ and a FOL query $q(\bar{x})$, the answers to q can be identified by refuting q w.r.t. $\mathcal{D}$: deriving `false` from $\mathcal{D} \cup \{\neg q(\bar{x})\}$ using resolution.
For instance, compute the answers to the queries $q_1(x)$ and $q_2(x_1, x_2)$ against the above database under OWA.

Data management challenges

Main data management challenges

1. Define data models and query languages of practical interests
2. Study the decidability and complexity of the data management problems [theory]
3. Devise effective/efficient solutions for the data management problems [algorithmics]